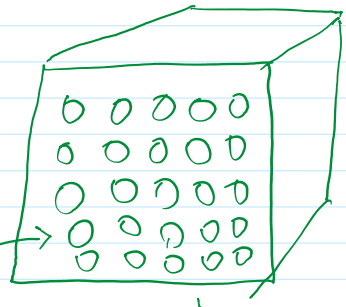
The 3 Cursors of Java

18 March 2025 23:52



1. Enumeration.
2. Iterator
3. List Iterator

} Cursors.

If you want to get back objects one-by-one from the collection. then we should go for cursors. There are 3 types of cursors.

1. Enumeration.
2. Iterator
3. List Iterator

Enumeration

We can use enumeration to get objects one by one from legacy collection object. We can create enumeration object by using elements () method of vector class.

```
Enumeration e = v.elements();
```

↑ vector object

Methods:

1. **public boolean hasMoreElements();**
2. **public Object nextElement();**

Official doc:

All Known Subinterfaces:
NamingEnumeration<T>

All Known Implementing Classes:
StringTokenizer

---

public interface **Enumeration<E>**

An object that implements the Enumeration interface generates a series of elements, one at a time. Successive calls to the `nextElement` method return successive elements of the series.

For example, to print all elements of a `Vector<E> v`:

```
for (Enumeration<E> e = v.elements(); e.hasMoreElements();)
    System.out.println(e.nextElement());
```

Methods are provided to enumerate through the elements of a vector, the keys of a hashtable, and the values in a hashtable. Enumerations are also used to specify the input streams to a `SequenceInputStream`.

NOTE: The functionality of this interface is duplicated by the Iterator interface. In addition, Iterator adds an optional remove operation, and has shorter method names. New implementations should consider using Iterator in preference to Enumeration.

Since:
JDK1.0

See Also:
Iterator, SequenceInputStream, nextElement(), Hashtable, Hashtable.elements(), Hashtable.keys(), Vector, Vector.elements()

Method Summary

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method and Description
boolean	**hasMoreElements()** Tests if this enumeration contains more elements.
E	**nextElement()** Returns the next element of this enumeration if this enumeration object has at least one more element to provide.

```java
import java.util.Enumeration;
import java.util.Vector;
public class EnumerationDemo{
    public static void main(String[] args) {
        Vector v = new Vector<>();
        for(int i = 0; i <= 10; i++){
            v.addElement(i);
        }
        System.out.println(v);
        for(Enumeration e = v.elements(); e.hasMoreElements();){
            Integer x = (Integer)e.nextElement();
            if(x % 2 == 0){
                System.out.println(x);
            }
        }
    }
}
```

Output:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
0
2
4
6
8
10
```

Limitations of Enumeration

1. We can apply enumeration concept only for legacy classes and it is not a universal cursor.
2. By using enumeration, we can get only read access. We cannot perform remove operation.
3. In enumeration, we can move in 1DIRECTION only i.e., forward direction.

Iterator

We can apply iterator concept for any collection object and hence it is a universal cursor.
By using iterator, we can perform both read and remove operations.
We can create iterator object using iterator () method.

Iterator iter = c.iterator();

↓Any collection object.

1. public boolean hasNext()
2. public Object next()
3. public void remove()

```
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorDemo {
    public static void main(String[] args) {
        ArrayList al = new ArrayList<>();
        for(int i = 0; i < 10; i++){
            al.add(i);
        }
        System.out.println("\nList: "+ al);
        Iterator iter = al.iterator();
        while (iter.hasNext()) {
            Integer x = (Integer)iter.next();
            if(x % 2 == 0){
                System.out.println(x);
            }
            else{
                iter.remove();
            }
        }
        System.out.println("\nUpdated list: "+al);
    }
}
```

Output:

```
List: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
0
2
4
6
8

Updated list: [0, 2, 4, 6, 8]
```

* Limitations of Iterator

1. Forward direction movement is applicable only. We can't move in backward direction. Enumeration (I) & Iterator (I) both are single direction cursor.

2. By using Iterator we can't perform replacement and addition of new object.

To overcome the above limitations

we should go for ListIterator(I).

List Iterator

By using ListIterator we can move either to the forward direction or to the backward direction and hence it is bi-directional cursor.

By using list iterator we can perform replacement and addition of new objects in. addition to read and remove operations.

Iterator (I)
↑
ListIterator (I)

ListIterator is a child interface of Iterator interface. And hence all methods present in the iterator interface by default available to the list iterator.

Interface ListIterator<E>

All Superinterfaces:
Iterator<E>

```
public interface ListIterator<E>
extends Iterator<E>
```

An iterator for lists that allows the programmer to traverse the list in either direction, modify the list during iteration, and obtain the iterator's current position in the list. A `ListIterator` has no current element; its *cursor position* always lies between the element that would be returned by a call to `previous()` and the element that would be returned by a call to `next()`. An iterator for a list of length n has n+1 possible cursor positions, as illustrated by the carets (^) below:

```
                      Element(0)   Element(1)   Element(2)   ... Element(n-1)
cursor positions:  ^            ^            ^            ^                  ^
```

Note that the `remove()` and `set(Object)` methods are *not* defined in terms of the cursor position; they are defined to operate on the last element returned by a call to `next()` or `previous()`.

This interface is a member of the Java Collections Framework.

Since:
1.2

public ListIterator listIterator()

eg: ListIterator litr = obj.listIterator();

Any Collection object which List property.

public boolean hasNext()
public Object next()
public int nextIndex()
} Forward movement

public boolean hasPrevious()
public Object previous()
public int previousIndex()
} Backward movement

public void remove()
public void add(Object o)
public void set(Object o)
} delete, add, modify or update
it replaces current object with ob.

Method Summary

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method and Description
void	add(E e) Inserts the specified element into the list (optional operation).
boolean	hasNext() Returns true if this list iterator has more elements when traversing the list in the forward direction.
boolean	hasPrevious() Returns true if this list iterator has more elements when traversing the list in the reverse direction.
E	next() Returns the next element in the list and advances the cursor position.
int	nextIndex() Returns the index of the element that would be returned by a subsequent call to next().
E	previous() Returns the previous element in the list and moves the cursor position backwards.
int	previousIndex() Returns the index of the element that would be returned by a subsequent call to previous().
void	remove() Removes from the list the last element that was returned by next() or previous() (optional operation).
void	set(E e) Replaces the last element returned by next() or previous() with the specified element (optional operation).

Example:

```
import java.util.*;
public class ListIteratorDemo {
    public static void main(String[] args) {
        LinkedList ll = new LinkedList();
        ll.add("Balakrishna");
        ll.add("Rama");
        ll.add("Ravichandran");
        ll.add("Lakshminarayan");
        ll.add("Bharat");
        System.out.println(ll);
        ListIterator li = ll.listIterator();
        while(li.hasNext()){
            String s = (String)li.next();
            if(s.equals("Rama")){
                li.remove();
            }
            else if(s.equals("Bharat")){
                li.add("Shiv");
            }
            else if(s.equals("Lakshminarayan")){
                li.set("LN");
            }
            System.out.println(ll+"\n");
        }
    }
}
```

Output:

```
[Balakrishna, Rama, Ravichandran, Lakshminarayan, Bharat]
[Balakrishna, Rama, Ravichandran, Lakshminarayan, Bharat]

[Balakrishna, Ravichandran, Lakshminarayan, Bharat]

[Balakrishna, Ravichandran, Lakshminarayan, Bharat]

[Balakrishna, Ravichandran, LN, Bharat]

[Balakrishna, Ravichandran, LN, Bharat, Shiv]
```

Note: The most powerful cursor is ListIterator, but its limitation is it is applicable only for List objects.

Comparison Table for 3 Cursors

Properties	Enumeration	Iterator	ListIterator
Where we can apply?	Only for legacy classes.	For any Collection object.	Only for list objects.
Is it legacy?(1.0V)	Yes	No (1.2V)	No(1.2V)
Movement	Single direction (only forward direction.)	Single direction (only forward direction.)	Bidirectional.
Allowed Operations	Only read.	Read and remove.	Read, remove, replace, add and update.
How we can get?	By using elements() of Vector class.	By using iterator() of Collection interface.	By using listIterator() method of List interface.
Methods	2-Mothods, hasMoreElements() nextElement()	3-methods hasNext() next() remove()	9 methods hasNext(), next(), nextIndex(), hasPrevious(), previous(), previousIndex(), remove(), add(Object o), set(Object o)

Inner Implementation of Cursor

```java
import java.util.Enumeration;
import java.util.Iterator;
import java.util.ListIterator;
import java.util.Vector;
public class InnerImplementationOfCursor {
    public static void main(String[] args) {
        Vector v = new Vector();
        Enumeration e = v.elements();
        Iterator itr = v.iterator();
        ListIterator litr = v.listIterator();
        System.out.println(e.getClass().getName());
        System.out.println(itr.getClass().getName());
        System.out.println(litr.getClass().getName());
    }
}
```

Output:

```
java.util.Vector$1
java.util.Vector$Itr
java.util.Vector$ListItr
```

inner class which is anonymous.

Iterator interface is implement in this inner class.

ListIterator interface is implemented

```
java.util.Vector$ListItr
```

ListIterator interface is implemented in this inner class.

```
public class Vector ....
{
    public class $1 implements Enumeration..
        @override
        hasMoreElement(){ }
        nextElement() { }
    }

    pub
    .
    .
    .
    .
```